ResumeLens: Screening Résumés with Formal Language Models

Four formal models, one pipeline: from raw résumé text to a validated candidate profile.

Juan Manuel Diaz Moreno (A00394477), Team Flux Computación y Estructuras Discretas III, Universidad Icesi · 2026-2 · Integrative Task 1

1 Introduction and Research Question

Recruiters review many résumés, and the same skill appears in many spellings: `JS`, `Javascript`, `JavaScript`. Before deciding whether a candidate fits a profile, the relevant text has to be found, made uniform, and compared against a precise pattern.

Formal language theory offers a model for each of these steps (Hopcroft, Motwani & Ullman, 2006). Regular expressions and transducers are established tools for text processing (Karttunen et al., 1996; Mohri, 1997), and context-free grammars define structured languages (Chomsky, 1956).

Research question

Can a pipeline built only from formal language models decide, transparently and reproducibly, whether a résumé satisfies the explicit qualification pattern of a professional profile?

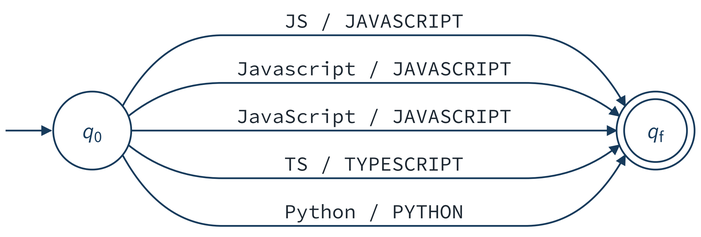
Scope: ResumeLens does not rank candidates or make hiring decisions.

2 Methods

1. Regular expressions extract the qualification categories.
2. Six finite-state transducers (FST), M = (Q, Σ, Γ, δ, ω, q0, F), map each spelling to one of 51 canonical terms.
3. Each profile is a deterministic finite automaton (DFA), M = (Q, Σ, δ, q0, F). Stage 2 sorts the input and the categories are disjoint, so every (state, symbol) pair has at most one next state; a nondeterministic automaton (NFA or NFA-λ) is not needed.
4. A domain-specific language (DSL), defined in Extended Backus–Naur Form (EBNF) and implemented with textX, describes the candidate.

Implementation: Python, pyformlang, textX, pytest.

Figure 2. Stage 2: programming-language transducer (excerpt), labeled input / output.



One transition per spelling; 13 transitions in total.

Figure 1. The ResumeLens pipeline.

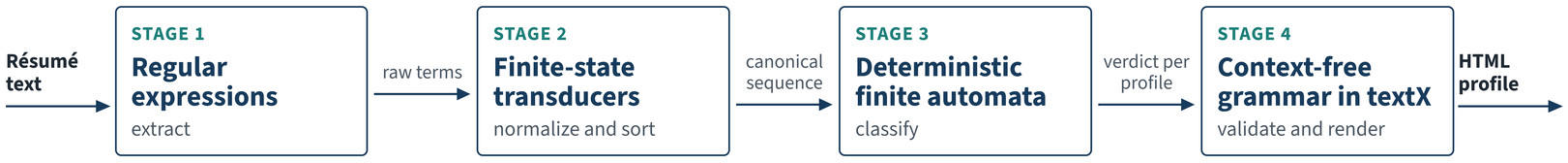
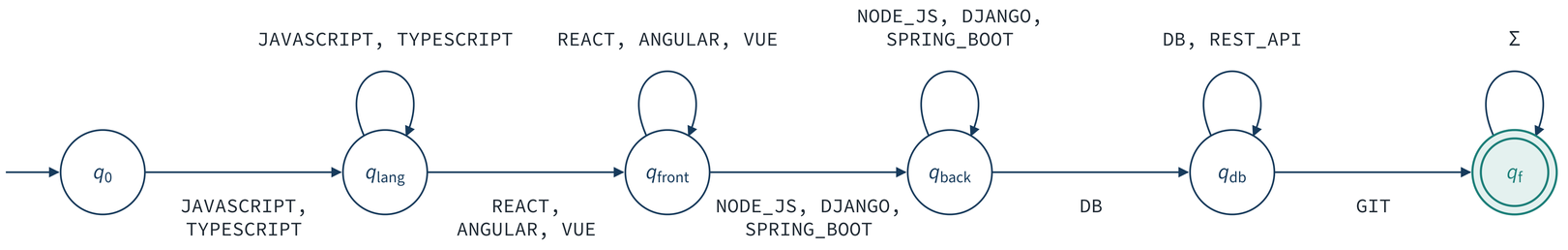


Figure 3. Stage 3: Full Stack Developer automaton (DFA).



`DB = {SQL, NOSQL, POSTGRESQL, MYSQL, MONGODB, SQLITE, REDIS, CASSANDRA}`. Missing symbols mean rejection.

3 Results and Findings

Figure 4. One résumé, end to end (Wednesday Addams, from the assignment).

Input	Technical Skills: JS, React.js, NodeJS, Postgres, Git.
1 Extract	JS, React.js, NodeJS, Postgres, Git
2 Normalize	`JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT`
3 Classify	`q0 → q_lang → q_front → q_back → q_db → qf` **Full Stack Developer ACCEPTED**

4 Describe	`candidate "Wednesday Addams"`
	`experience 3 years`
	`skills JAVASCRIPT REACT NODE_JS POSTGRESQL GIT`
	`classification FULL_STACK_DEVELOPER ACCEPTED …`

Figure 6. Validation of the candidate language.

Level	Example	Result
Lexical	`skill written python`	rejected: does not match `[A-Z][A-Z0-9_]*`
Syntactic	skills section missing	rejected: violates the grammar
Semantic	`skill COBOL`	rejected: not in the canonical vocabulary

Figure 5. Classification of ten sample résumés (✔ = accepted, blank = rejected).

	Full Stack	ML Engineer	DevOps	Data Engineer
Wednesday Addams	✔			
Peter Parker	✔			
Daenerys Targaryen	✔		✔	
Mary Jane Watson		✔		
Eleven		✔		✔
Tony Stark			✔	
Michael Scott			✔	
Hermione Granger				✔
Walter White				✔
Rick Sanchez				

Findings

- Both worked examples from the assignment are reproduced exactly.
- 7 résumés match one profile, 2 match two profiles, and 1 matches none.
- The verdict does not depend on the order in which skills are written.
- All 99 automated tests pass.

4 Conclusions

Every step of résumé screening maps to a formal model, and the right preprocessing lets the simplest model do the job. Sorting in stage 2 is what lets stage 3 be deterministic.

Limitations

- The vocabulary is fixed.
- Only ten hand-written résumés were tested.

Future research

- A larger labeled résumé set, to measure precision and recall.
- Experience parsed per job.

References. Chomsky, N. (1956). IRE Trans. Information Theory, 2(3). · Dejanović, I. et al. (2017). Knowledge-Based Systems, 115. · Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation. · Karttunen, L. et al. (1996). Natural Language Engineering, 2(4), 305–328. · Mohri, M. (1997). Computational Linguistics, 23(2), 269–311.

Acknowledgments. AI assistance (Claude Code) was used and is documented in the repo. Code: `github.com/Juanmadiaz45/resumelens`